

研究方法整理：從未知到實驗設計

根據我們的對話整理成 Markdown 與 PDF 筆記

ChatGPT 與使用者對話整理

2026-07-08

目录

1	這份筆記的目的	2
2	研究的核心邏輯	2
2.1	從「想證明」改成「想知道」	2
2.2	每個實驗前先寫三句話	3
3	正確做研究的循環	3
4	研究問題、假設與研究缺口	4
4.1	研究問題 Research Question	4
4.2	假設 Hypothesis	4
4.3	研究缺口 Research Gap	4
5	三種變因	5
5.1	操縱變因 Independent Variable	5
5.2	應變變因 Dependent Variable	5
5.3	控制變因 Control Variable	5
6	Baseline 與 Ablation	6
6.1	Baseline 是比較對象	6
6.2	Ablation Study 是拆解你的方法	6
7	圖和表格的意義	7
7.1	表格：回答「誰比較好？」	7
7.2	折線圖：回答「參數改變會怎樣？」	7
7.3	散佈圖：回答「兩個變數有沒有關係？」	8
7.4	熱力圖 Heatmap：回答「哪裡比較敏感？」	8
7.5	Case Study：回答「失敗案例長什麼樣？」	8
8	不要重複實驗，要往下切	9
8.1	往下切的層次	9
8.2	每次只改一個主要因素	9
9	根據別人的實驗修正方法	9
9.1	正確做法	10
9.2	要避免 Cherry-picking 與 P-hacking	10

10 用 LLM Serving 題目示範完整研究流程	10
10.1 Experiment 1：先驗證核心假設	10
10.2 Experiment 2：檢查 router 是否有用	11
10.3 Experiment 3：分析 router 錯在哪裡	11
10.4 Experiment 4：加入 fallback	12
10.5 Experiment 5：檢查 serving 效率	12
11 每次實驗後要寫 Decision	12
12 研究筆記模板	13
13 論文常見結構	14
14 研究倫理與客觀性檢查	14
14.1 不要只挑漂亮結果	14
14.2 不要一直用 test set 調方法	15
14.3 不要讓方法變得越來越複雜卻不知道原因	15
15 最後的檢查清單	15
16 一句話總結	15

1 這份筆記的目的

這份筆記整理我們前面討論過的研究方法。重點不是把聊天內容逐字抄下來，而是把它整理成一份可以拿來寫論文、讀論文、設計實驗、解讀圖表的研究工作筆記。

核心觀念可以濃縮成一句話：

正確做研究不是先想「我要證明我是對的」，而是先問「我目前不知道什麼」，再設計公平的實驗去回答這個未知。

所以，做研究的流程不是：

我有一個方法 -> 跑實驗 -> 找漂亮數字 -> 說我的方法很好。

比較客觀的流程應該是：

我有一個未知 -> 我提出假設 -> 我設計實驗 -> 我用圖表回答問題 -> 我根據結果決定下一步。

2 研究的核心邏輯

2.1 從「想證明」改成「想知道」

研究中很容易犯的錯，是一開始就想證明自己的方法比較好。這會讓人不自覺只看支持自己的結果，忽略失敗或反例。

比較好的研究態度是：

我懷疑 A 會影響 B，所以我要檢查 A 和 B 的關係是否真的存在。

例如在 LLM serving 與 precision routing 的方向中，不要一開始就說：

我要證明我的 router 有用。

比較好的說法是：

我不知道不同 query 對低精度推論的敏感度是否不同，所以我想檢查 query difficulty 和 low-precision quality drop 是否有關。

這樣你的研究會比較客觀，因為你是在回答未知，而不是硬要證明自己是對的。

2.2 每個實驗前先寫三句話

任何實驗開始前，先寫清楚這三句：

1. 我原本不知道什麼？
2. 我想看到什麼現象？
3. 這張圖或這個表要回答什麼問題？

例如：

問題	例子
原本不知道什麼	我不知道 query difficulty 是否能預測 INT4 下的品質下降。
想看到什麼	如果 hard query 在 INT4 下比 easy query 掉更多品質，代表 query-aware router 有合理性。
圖表回答什麼	scatter plot 要回答 difficulty score 和 quality drop 是否有正相關。

這樣做可以避免你只是為了做圖而做圖。

3 正確做研究的循環

客觀研究可以看成一個循環：

Research Question
-> Hypothesis
-> Experiment Design
-> Figure / Table
-> Interpretation
-> Decision
-> Finer Question
-> Next Experiment

中文來說：

研究問題
-> 假設
-> 實驗設計
-> 圖表證據
-> 解讀結果
-> 做出決策

- > 形成更細的問題
- > 下一輪實驗

重點是，實驗不是結束在「數字是多少」，而是要結束在「這個結果讓我下一步該怎麼做」。

4 研究問題、假設與研究缺口

4.1 研究問題 Research Question

研究問題是在問：現有方法有什麼問題？還有什麼未知？

例如：

Can query-aware precision routing improve LLM serving throughput while maintaining output quality?

中文意思是：

能不能根據每個 request 的難易度，動態選擇推論精度，讓吞吐量提高，同時不要讓回答品質下降太多？

好的研究問題通常要能讓人看出三件事：

面向	問題
問題的重要性	為什麼這件事值得研究？
現有方法限制	別人已經做了什麼，還缺什麼？
可驗證性	這個問題能不能用實驗回答？

4.2 假設 Hypothesis

假設是你相信可能成立、但還需要實驗驗證的事情。

例如：

不同 query 對模型精度的需求不同。簡單 query 可以用較低精度，困難 query 則需要較高精度。因此 query-aware precision routing 可能可以節省計算成本，同時維持品質。

這個假設後面要靠實驗驗證。假設如果不成立，也是一個有價值的研究結果，因為它幫你排除了一個錯誤方向。

4.3 研究缺口 Research Gap

研究缺口是：前人方法還沒有解好的地方。

以 LLM serving 為例，可能的 research gap 是：

現有 serving 方法常常只考慮 batching、request 數量或固定 precision，較少根據 query 的難度、品質需求與延遲限制來動態分配 precision budget。

研究貢獻不是「我做了一個 router」而已，而是要說清楚：

前人方法少考慮什麼 -> 這造成什麼問題 -> 我的方法如何補上這個缺口 -> 實驗如何證明真的改善。

5 三種變因

你前面提到的「控制變因、應變變因」是研究設計的基礎。完整來說，常見有三種變因。

5.1 操縱變因 Independent Variable

操縱變因，也可以叫自變因，是你主動改變的東西。

例如：

操縱變因	例子
precision policy	FP16、INT8、INT4、mixed precision
routing method	no router、rule-based router、learned router
batching method	normal batching、precision-aware batching
fallback threshold	0.1、0.2、0.3、0.5
query 類型	easy、medium、hard

研究上要問的是：當我改變這個因素，結果會不會跟著變？

5.2 應變變因 Dependent Variable

應變變因是你觀察的結果，也就是 metric。

例如：

應變變因	意義
latency	每個 request 完成需要多久
throughput	每秒能處理多少 requests 或 tokens
accuracy / F1 / BLEU / ROUGE	回答品質或任務分數
quality drop	低精度相對高精度掉了多少品質
memory usage	GPU 記憶體使用量
GPU utilization	GPU 是否被有效使用
QoS violation rate	有多少 request 沒達到品質或延遲要求
fallback rate	有多少 request 被升級到較高精度

簡單說，應變變因就是你想看的結果。

5.3 控制變因 Control Variable

控制變因是你不希望它亂變的東西。因為如果太多因素同時改變，你就不知道結果差異到底是誰造成的。

例如比較 FP16 和 INT4 時，應該固定：

控制變因	為什麼要固定
model	避免模型不同造成差異
dataset	避免題目難度不同

控制變因	為什麼要固定
hardware	避免 GPU 不同造成速度差
batch size	避免 batching 效果混進結果
max tokens	避免輸出長度不同影響 latency
prompt template	避免 prompt 改變造成品質差異
decoding setting	temperature、top-p 會影響輸出
random seed	避免隨機性造成結果不穩

控制變因的目的就是：讓結果差異盡量只來自你想研究的因素。

6 Baseline 與 Ablation

6.1 Baseline 是比較對象

論文不能只說「我的方法很快」或「我的方法品質很好」，必須回答：跟誰比？好多少？

常見 baseline 有：

Baseline 類型	例子	目的
原始方法	FP16 inference	作為品質上限或標準做法
固定低精度	INT8、INT4	看低精度速度與品質的 trade-off
簡單方法	random routing	證明不是隨便分配也有效
現有方法	前人論文方法	證明你的方法相對現有研究有價值
去掉模組	no fallback、no batching	檢查你的模組是否真的有用

你的方法如果是 query-aware precision routing，可以有這些 baseline：

1. FP16 full precision
2. fixed INT8
3. fixed INT4
4. random precision routing
5. query-aware routing only
6. query-aware routing + precision-aware batching
7. query-aware routing + batching + fallback，也就是完整方法

6.2 Ablation Study 是拆解你的方法

Ablation study 的意思是：把方法中的某個部件拿掉，看結果是否變差。

例如你的完整方法有三個模組：

1. query difficulty router
2. precision-aware batching
3. fallback mechanism

可以設計這樣的 ablation table：

Method	Router	Batching	Fallback	Throughput	Quality
FP16 baseline	no	no	no	1.0x	100%
Router only	yes	no	no	1.4x	96%
Router + batching	yes	yes	no	1.8x	95%
Full method	yes	yes	yes	1.7x	98%

這張表不是只看誰最高，而是在說明每個零件的作用：

- router 可能讓速度變快。
- batching 可能進一步提高 throughput。
- fallback 可能犧牲一點速度，但救回品質。

7 圖和表格的意義

你前面提到「圖和表格的意義」，這其實是論文閱讀與寫作的核心能力。

圖表不是裝飾，而是證據。每張圖表都要回答一個明確問題。

7.1 表格：回答「誰比較好？」

表格通常用來呈現精確數字，常見於 main result 或 ablation study。

例如：

Method	Latency ↓	Throughput ↑	Quality ↑
FP16	100 ms	1.0x	90.0
INT4	55 ms	1.9x	82.0
Ours	65 ms	1.6x	89.0

這張表想回答：

Ours 是否在速度與品質之間取得比 baseline 更好的 trade-off？

注意，Ours 不一定要在所有欄位都是第一名。研究常常是在處理 trade-off。例如 INT4 最快但品質差，FP16 品質最好但慢，Ours 的價值可能是品質接近 FP16，同時速度明顯比 FP16 快。

7.2 折線圖：回答「參數改變會怎樣？」

折線圖常用來看 threshold、batch size、precision budget 改變時，結果如何變化。

例如：

Fallback threshold	Throughput	Quality
0.1	high	lower
0.3	medium	medium
0.5	lower	high

這種圖回答：

fallback 越積極，品質是否變好？代價是不是 latency 或 throughput 變差？

這是在看參數敏感度與 trade-off。

7.3 散佈圖：回答「兩個變數有沒有關係？」

散佈圖常用來看相關性。

例如：

- x 軸：query difficulty score
- y 軸：INT4 相對 FP16 的 quality drop

這張圖要回答：

query 越難，低精度造成的品質下降是否越大？

如果沒有明顯關係，就代表 query difficulty 可能不是好的 router feature，或需要加入其他 feature，例如 uncertainty、entropy、confidence、retrieval score。

7.4 熱力圖 Heatmap：回答「哪裡比較敏感？」

熱力圖常用在 layer sensitivity、precision allocation、error distribution。

例如：

Layer	INT4 error	INT8 error
Layer 1	low	low
Layer 20	high	medium
Layer 31	high	high

這種圖回答：

哪些 layer 對低精度最敏感？是否有必要做 layer-wise mixed precision？

7.5 Case Study：回答「失敗案例長什麼樣？」

數字可以告訴你整體結果，case study 可以幫你理解錯誤型態。

例如你可以比較：

Query	Router decision	Correct precision	Error type
簡單事實問答	INT4	INT4	correct
多步推理題	INT4	FP16	hard query 被低估
長上下文摘要	INT8	FP16	context sensitivity

這種表格回答：

router 主要錯在哪裡？是 hard query 被誤判成 easy，還是某些任務類型特別容易失敗？

8 不要重複實驗，要往下切

你提到「如果已經做過一次了，下次實驗就不要重複，要再分細一點」，這非常重要。

第一次實驗可能回答大問題：

Ours 有沒有比 baseline 好？

如果已經知道整體上有效，下一輪就不應該只是重複同一張 main result table，而要問更細的問題。

8.1 往下切的層次

層次	問題
第一層：整體有效嗎	Ours 是否比 FP16、INT8、INT4 更好？
第二層：為什麼有效	是 router、batching 還是 fallback 造成改善？
第三層：在哪裡有效	easy、medium、hard query 是否都有幫助？
第四層：什麼時候失效	哪些 query 或 dataset 會讓方法失敗？
第五層：參數敏感嗎	threshold、batch size、precision budget 改變後結果穩不穩？
第六層：能否泛化	換 model、dataset、hardware 後是否還成立？

8.2 每次只改一個主要因素

不要同時改 router feature、fallback threshold、batching policy、dataset、decoding setting。否則結果變好或變壞，你不知道原因。

比較乾淨的實驗順序是：

Round	只改什麼	看什麼
1	fixed precision	建立 FP16、INT8、INT4 baseline
2	加 router	看 quality-speed trade-off 是否改善
3	加 fallback	看 quality 是否被救回來
4	加 batching	看 throughput 是否提升
5	改 threshold	看參數敏感度
6	換 dataset 或 model	看泛化能力

這就是控制變因在實際研究中的用法。

9 根據別人的實驗修正方法

你問過：「隨著別人的實驗、實作、好的實驗數據再去修改這個方法呢？」答案是可以，而且這是正常研究迭代。

但不能只是看到哪個數據漂亮就改成那樣。比較正確的說法是：

根據前人實驗發現與實作限制，修正自己的假設、方法設計與實驗設定。

9.1 正確做法

前人或自己看到的現象	可以怎麼修正方法
hard query 在低精度下品質掉比較多	設計 query difficulty router
某些 layer 對 quantization error 特別敏感	做 layer-wise precision allocation
batch 內 precision 太分散會降低效率	做 precision-aware batching
router 有時誤判 hard query	加 uncertainty-based fallback
INT4 很快但品質掉太多	改成 mixed precision 或 selective fallback

這不是抄襲，而是研究累積。但你必須說清楚：

1. 你從前人結果中觀察到什麼限制？
2. 你的方法如何針對這個限制改進？
3. 你的實驗如何證明這個改進有效？

9.2 要避免 Cherry-picking 與 P-hacking

不好的做法是：

我先亂試很多設定，哪個結果最好，就說我的方法本來就是這樣設計的。

這會讓 reviewer 質疑：

- threshold 為什麼是這個值？
- router feature 為什麼選這些？
- 是否只挑了對你有利的 dataset？
- 換模型後是否還成立？
- test set 是否被你反覆調參調到過擬合？

比較安全的做法是把資料分成：

資料	用途
train set	訓練 router 或 predictor
validation set	調 threshold、feature、fallback 參數
test set	最後評估，只用來報告最終結果

你可以反覆使用 validation set 修方法，但不要一直看 test set 改方法。

10 用 LLM Serving 題目示範完整研究流程

以下用我們前面討論的 query-aware precision routing 當例子。

10.1 Experiment 1：先驗證核心假設

Question：

不同 query 對低精度是否真的有不同敏感度？

Setup：

- 固定 model、dataset、hardware、prompt、decoding setting。
- 同一批 query 分別跑 FP16 與 INT4。
- 計算每個 query 的 quality drop。

Figure :

- x 軸：query difficulty score
- y 軸：INT4 相對 FP16 的 quality drop

Expected pattern :

difficulty 越高，quality drop 越大。

Decision :

- 如果相關性明顯，query difficulty 可以作為 router feature。
- 如果相關性很弱，需要加入 uncertainty、entropy、confidence 或其他 feature。

10.2 Experiment 2：檢查 router 是否有用

Question :

query-aware router 是否比 fixed precision 更好？

Table :

Method	Quality	Latency	Throughput
FP16	high	slow	low
INT8	medium-high	medium	medium
INT4	low	fast	high
Random router	unknown	unknown	unknown
Query-aware router	target: close to FP16	target: faster	target: higher

Interpretation :

如果 router 的品質接近 FP16，但 latency 明顯低於 FP16，就代表 routing 有價值。

10.3 Experiment 3：分析 router 錯在哪裡

Question :

router 失敗的 case 是什麼？

Possible figures :

- confusion matrix
- easy / medium / hard 分組表
- case study table

想知道的是：

- router 是否常把 hard query 判成 easy？
- medium query 是否最難分？
- 長上下文、多步推理、數學題是否特別容易失敗？

這一步比 Experiment 2 更細，因為它不是再問「有沒有比較好」，而是在問「為什麼還會錯」。

10.4 Experiment 4：加入 fallback

Question：

fallback 是否能降低 router 誤判造成的品質下降？

Figures：

- threshold vs quality
- threshold vs latency
- threshold vs fallback rate

Expected pattern：

fallback rate 不需要太高，但可以顯著救回 quality。

Interpretation：

router 負責省計算，fallback 負責保品質。

10.5 Experiment 5：檢查 serving 效率

Question：

precision-aware batching 是否真的提升 throughput？

Table：

Batch policy	Throughput	GPU utilization	Quality
Normal batching	baseline	baseline	same
Precision-aware batching	higher	higher	same

Interpretation：

如果 quality 不變，但 throughput 與 GPU utilization 變好，代表 batching 模組有 serving-aware 的貢獻。

11 每次實驗後要寫 Decision

做完實驗後，不要只記錄數字。你要寫出這個結果代表什麼，以及下一步要做什麼。

建議每次實驗用這個格式記錄：

欄位	內容
Question	這次要回答什麼問題？
Setup	固定了哪些條件？
Changed Variable	這次只改了什麼？
Metrics	觀察哪些指標？
Figure / Table	用什麼圖表呈現？
Result	看到什麼數字或趨勢？

欄位	內容
Interpretation	這代表什麼？
Decision	下一步要做什麼？
Risk	這個結論可能哪裡不穩？

範例：

欄位	例子
Question	query difficulty 是否能預測 INT4 品質下降？
Setup	固定 Qwen2.5-7B、同 dataset、同 decoding、同 max tokens。
Changed Variable	query difficulty score。
Metrics	quality drop、correlation。
Figure / Table	difficulty vs quality drop scatter plot。
Result	correlation 只有 0.12。
Interpretation	單靠 difficulty 不夠。
Decision	加入 entropy、confidence 或 output uncertainty。
Risk	quality metric 可能太粗，沒有抓到語意錯誤。

12 研究筆記模板

你可以把每個研究 idea 都先寫成下面格式。

Experiment Name

Unknown

我目前不知道什麼？

Hypothesis

我猜測什麼？

Setup

我要固定哪些東西？

Changed Variable

我這次只改什麼？

Metrics

我要看哪些指標？

Figure / Table

我要用什麼圖或表回答問題？

Expected Pattern

如果假設成立，我應該看到什麼？

Result

實驗結果是什麼？

Interpretation

結果代表什麼？

Decision

下一步要做什麼？

Risk / Limitation

這個結論可能哪裡不穩？

這個模板的目的，是強迫自己每次實驗都連回「未知、假設、證據、決策」。

13 論文常見結構

大部分 CS / ML 論文可以整理成：

Section	作用
Abstract	一段話總結問題、方法、結果
Introduction	說明問題重要性、gap、貢獻
Related Work	說別人做過什麼，你跟他們差在哪
Background	解釋必要知識，例如 quantization、serving、batching
Method	你的方法
Experimental Setup	模型、資料集、硬體、metrics、baseline
Results	主要結果
Ablation / Analysis	拆解方法，解釋為什麼有效
Limitations	說明方法限制
Conclusion	總結研究發現

你讀論文時，可以先抓三件事：

1. 這篇解決什麼問題？
2. 它的方法核心是什麼？
3. 它如何證明自己有效？

14 研究倫理與客觀性檢查

14.1 不要只挑漂亮結果

如果某些 dataset 結果不好，也要記錄。你可以分析原因，而不是直接刪掉。

負結果也有價值，例如：

query length 和 precision sensitivity 幾乎無關。

這代表 query length 可能不是好的 router feature，這可以幫你避免走錯方向。

14.2 不要一直用 test set 調方法

可以用 validation set 反覆調參，但 test set 應該保留到最後。否則你可能不是方法真的泛化，而是把 test set 調到過擬合。

14.3 不要讓方法變得越來越複雜卻不知道原因

常見錯誤是：

結果不好 -> 加一個模組 -> 還不好 -> 再加一個 heuristic -> 最後方法很複雜，但不知道哪個東西真的有用。

比較好的方式是：

每次只改一個主要因素，並用 ablation 證明它的貢獻。

15 最後的檢查清單

做研究前，先問：

- 我現在真正不知道的是什麼？
- 我的 hypothesis 是什麼？
- 這個 hypothesis 可以被實驗反駁嗎？
- 我這次只改了一個主要因素嗎？
- 我固定了哪些控制變因？
- 我的 baseline 是否公平？
- 我的 metric 是否真的能回答研究問題？
- 這張圖或表到底要回答哪個問題？
- 如果結果出來，我下一步會根據它做不同決策嗎？
- 我有沒有避免 cherry-picking？
- 我有沒有避免 test set overfitting？
- 如果結果不好，我是否能從中學到下一步？

16 一句話總結

客觀做研究就是：

把「我想證明什麼」改成「我目前不知道什麼」；把「我要做什麼圖」改成「這張圖要回答什麼問題」；把「結果好不好」改成「這個結果讓我下一步該怎麼決策」。

對你的 LLM serving 題目來說，核心不是只說「我做了一個 router」，而是要證明：

1. query 真的有不同 precision 需求；
2. router 真的能分辨這件事；
3. 分辨後真的能改善 latency / throughput；
4. 品質下降是可控的；
5. batching 與 fallback 不是裝飾，而是真的有貢獻；
6. 你的方法在不同 dataset、model 或設定下仍然有一定穩定性。

做到這些，你的研究就會從「跟著別人跑實驗」變成「自己設計問題、驗證假設、解釋結果」。